

Facultad de Ciencias UNAM
Estructuras de Datos
Nota5: Recurrencias

Profesora: Virginia Teodosio Procopio
Ayudantes: Leonardo Gallo Guerrero
José Manuel Madrigal Ramírez

26 de Septiembre de 2026

Recurrencias

Hasta el momento solo hemos obtenido funciones de complejidad para algoritmos secuenciales. Pero los algoritmos recursivos son sumamente interesantes y aún nos queda pendiente el análisis de **MaxSum3**, el algoritmo recursivo que resolvía el problema de la suma máxima. Pero ¿cómo podemos cuantificar el tiempo de ejecución de un algoritmo que se llama a sí mismo?

Por suerte no estamos condenados a caer en un bucle *ad infinitum* ya que todo algoritmo recursivo cuenta con un caso base que se caracteriza por no contener llamadas al mismo algoritmo. Además, el parámetro con el que se realiza la llamada recursiva suele recibir cambios que se acumulan con cada llamada hasta que eventualmente desenvoca en una llamada al caso base, devolviendo un resultado y concluyendo la ejecución del algoritmo.

Resumiendo este par de observaciones podríamos considerar la siguiente estructura general para un algoritmo recursivo:

Algoritmo 1 Recursivo(n)

```
1: if casoBase( $n$ ) then  
2:   return resultadoBase  
3: else  
4:    $\vdots$   
5:    $s_1 \leftarrow \text{Recursivo}(n\_reducido_1)$   
6:    $s_2 \leftarrow \text{Recursivo}(n\_reducido_2)$   
7:    $\vdots$   
8: end if  
9: return Solucion( $s_1, s_2, \dots$ )
```

Parece natural definir su función de complejidad $T(n)$ por casos:

$$T(n) = \begin{cases} k & \text{casoBase}(n) \\ f(T(n_reducido)) & \text{e.o.c.} \end{cases} \quad (1.1)$$

Cuando n cumple la condición de ser caso base, contaremos un número $k \in \mathbb{N}$ constante de operaciones. En caso contrario, devolveremos el conteo de operaciones $T(n_reducido)$, sobre la entrada reducida según dicte el algoritmo. Hemos escrito $f(T(n_reducido))$ para indicar que el resultado dado por $T(n_reducido)$ aún puede modificarse dependiendo de la naturaleza del algoritmo, por ejemplo, como veremos a continuación, en **Factorial** la expresión para la función de complejidad en el caso recursivo es $T(n) = T(n) + c$, donde $c \in \mathbb{N}$.

A las expresiones del tipo 1.4 se les denomina **recurrencias**.

Factorial

A continuación tenemos el algoritmo para obtener $n!$:

Algoritmo 2 Factorial (n)

Precondiciones: n es un entero, $n \geq 0$

```
1: if  $n \leq 1$  then  
2:   return 1  
3: else  
4:    $i \leftarrow \text{Factorial}(n - 1)$   
5:   return  $n * i$   
6: end if
```

Siguiendo el patrón de 1.4 para recurrencias y dado que en cada llamada recursiva restamos una unidad al dato de entrada n tenemos:

$$T(n) = \begin{cases} b & \text{casoBase}(n) \\ T(n-1) + k & \text{e.o.c.} \end{cases} \quad (1.2)$$

Donde $b \in \mathbb{N}$ es una constante que indica la cantidad de operaciones realizadas al ejecutarse el caso base, a saber, la comparación y el retorno. Sin embargo, este número puede ser distinto si sólo consideramos la operación esencial en nuestra cuantificación, por ello escribimos b en vez de 2.

Ahora nos enfrentamos a un problema ¿cómo demostramos a qué orden de complejidad pertenece nuestra función $T(n)$?

Debemos transformar esta recurrencia a una función que no esté escrita en términos de sí misma, a este procedimiento se le llama *resolver la recurrencia*. Para ello existen tres métodos[1]:

- Inducción
- Solución de la ecuación característica
- Solución por sustitución

En estas notas utilizaremos la solución por sustitución para resolver nuestras recurrencias. Este método consiste, para sorpresa de nadie, en sustituir la expresión de recurrencia para casos reducidos, ($T(n-1)$ en el caso del factorial), en la expresión para el caso inicial n . Veamos:

$$\begin{aligned} T(n) &= T(n-1) + k && \text{expresión para el caso inicial} \\ T(n-1) &= T((n-1)-1) + k = T(n-2) + k && \text{expresión para el caso reducido} \\ T(n) &= (T(n-2) + k) + k = T(n-2) + 2k && \text{sustituyendo el caso reducido en el inicial} \end{aligned}$$

Repetiendo este procedimiento para $T(n-2)$:

$$\begin{aligned} T(n-2) &= T(n-3) + k \\ T(n) &= T(n-3) + 3k \end{aligned}$$

Y entonces podemos deducir un patrón escrito en términos de un índice i :

$$T(n) = T(n-i) + ik \quad (1.3)$$

¡Al fin las piezas se unen! Observa que si encontramos la forma de incrustar el caso base $T(1)$ en el lado derecho de la expresión 1.3 obtendríamos una expresión del estilo $T(n) = T(1) + ik$ para algún índice i , es decir, una expresión sin recursión.

Entonces, nos toca encontrar ese índice i , cosa que podemos lograr igualando el caso base $T(1)$ con el patrón correspondiente de la expresión 1.3, $T(n - i)$:

$$\begin{aligned} T(n - i) &= T(1) \\ \Rightarrow 1 &= n - i \\ \Rightarrow i &= n - 1 \end{aligned}$$

Con lo cual podemos escribir una expresión para $T(n)$ en términos del caso base:

$$\begin{aligned} T(n) &= T(1) + (n - 1) + k \\ &= b + nk - k \end{aligned}$$

Esta expresión es de orden $O(n)$. Queda como tarea moral encontrar una constante $c \in \mathbb{R}^+$ y un número $n_0 \in \mathbb{N}$ tal que para todo otro $n \in \mathbb{N}$ se cumpla que:

$$T(n) \leq cn$$

Con lo cual quedaría demostrado que este algoritmo para encontrar el factorial de un número n es de orden lineal $O(n)$.

Más ejemplos de resolución de recurrencias

A partir de lo dicho en la sección anterior podríamos deducir una serie de pasos para encontrar la función de complejidad de un algoritmo recursivo:

1. Definir la recurrencia
2. Sustituir suficientes veces para encontrar una expresión en términos de un índice i .
3. Escribir dicha expresión para el caso base

¡Encontremos la función de complejidad de los siguientes algoritmos recursivos!

Algoritmo 3 Foo (n)

Precondiciones: n es un entero, $n \geq 0$

```

1: if  $n \leq 1$  then
2:   return 1
3: else
4:   return  $2 * \text{Foo}(n/2)$ 
5: end if

```

1. *Definir la recurrencia:*

En esta ocasión nos concentraremos en las operaciones esenciales, por lo que asignaremos una sólo operación al caso base y en el caso recursivo añadiremos una operación por llamada. Además, observa que el cambio que reduce la variable de entrada en cada llamada recursiva es una división entre dos:

$$T(n) = \begin{cases} 1 & n = 0 \\ T(n/2) + 1 & e.o.c. \end{cases} \quad (1.4)$$

2. *Sustituir suficientes veces para encontrar una expresión en términos de un índice i :*

$$\begin{aligned} T(n) &= T(n/2) + 1 && \text{por definición de } T(n). \\ T(n/2) &= T(n/2^2) + 1 && \text{expresión para } n/2. \\ T(n) &= T(n/2^2) + 1 + 1 && \text{sustituyendo y reduciendo.} \\ T(n) &= T(n/2^2) + 2 \end{aligned}$$

Sustituimos una vez más para encontrar la expresión en términos de un índice i . Hemos dejado la expresión 2^2 sin reducir a propósito porque nos va a ser de ayuda:

$$\begin{aligned} T(n/2^2) &= T(n/2^3) + 1 && \text{expresión para } n/2. \\ T(n) &= T(n/2^3) + 1 + 1 + 1 && \text{sustituyendo en } T(n) \text{ y reduciendo.} \\ T(n) &= T(n/2^3) + 3 \end{aligned}$$

Observa las distintas expresiones para $T(n)$ ¿puedes encontrar el patrón? Parece que la potencia y el número que sumamos están relacionados y podríamos sugerir la siguiente expresión en términos de un índice i :

$$T(n) = T(n/2^i) + i \tag{1.5}$$

3. *Escribir dicha expresión para el caso base:*

A partir de 1.5 podríamos obtener un valor de i que nos permita tener 1 como parámetro de la función en el lado izquierdo, es decir, queremos $T(n) = T(1) + algo$.

Justamente, si sustituimos i por $\log_2 n$ obtenemos:

$$\begin{aligned} T(n) &= T(n/2^{\log_2 n}) + \log_2 n && \text{expresión para } n/2. \\ T(n) &= T(n/n) + \log_2 n && \text{sustituyendo en } T(n) \text{ y reduciendo.} \\ T(n) &= T(1) + \log_2 n \\ T(n) &= 1 + \log_2 n && \text{¡La expresión que buscábamos!} \end{aligned}$$

De modo que $T(n) = 1 + \log_2 n$ es la función de complejidad para nuestro algoritmo Foo(n). La intuición nos dice que pertenece al orden $O(\log n)$ pero ¿cómo lo justificamos?

De nuestras notas anteriores recordemos que la estrategia para demostrar que $f \in O(g)$ es dar con un múltiplo de cg de g que acote a f a partir de un punto $n_0 \in \mathbb{N}$, exhibiendo quién es $c \in \mathbb{R}^+$ y quién es n_0 .

Entonces hechamos mano de unas desigualdades mañosas, en nuestro caso podemos asegurar que:

$$1 + \log_2 n \leq \log_2 n + \log_2 n = 2\log_2 n$$

siempre que $n > 2$. Como podemos ver en la siguiente gráfica:

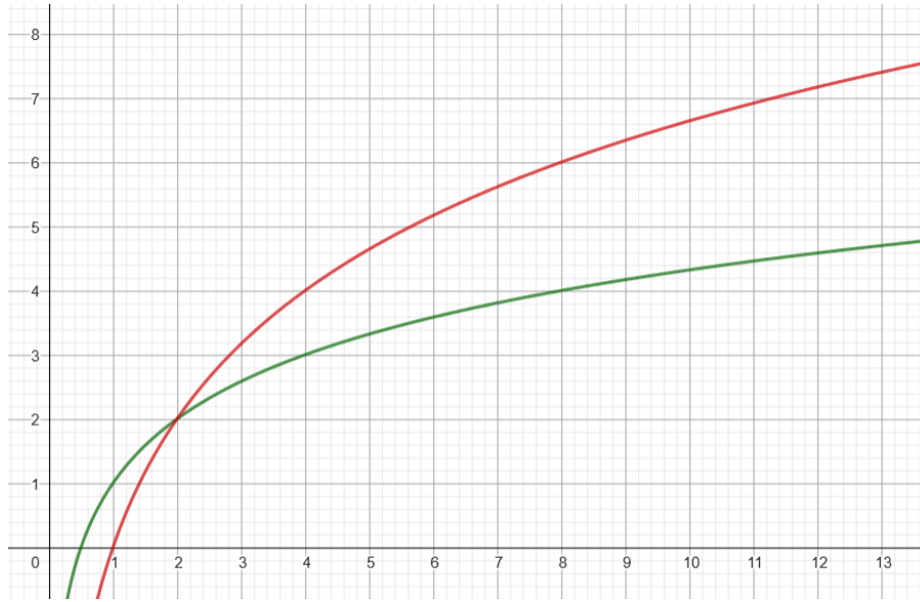


Figura 1: Gráfica de $1 + \log_2 n$ y $2 \log_2 n$

Por tanto, concluimos que para valores $n_0 = 2$ y $c = 2$ se cumple:

$$T(n) \leq c \log_2 n$$

Y concluimos que $T(n) \in O(\log n)$.

Algoritmo 4 Goo (n, x)

Precondiciones: n y x son enteros

```

1: if  $n \leq 1$  then
2:   return 1
3: else
4:   for  $i \in \{1, \dots, n\}$  do
5:      $x \leftarrow x + 1$ 
6:   end for
7:   return Goo( $n - 1, x$ )
8: end if
```

Ahora nos toca encontrar la función de complejidad para el algoritmo $Goo(n, x)$:

1. Definir la recurrencia

Nuevamente para el caso base contaremos una operación elemental, sin embargo, en el caso recursivo nos encontramos un ciclo **for**. Este ciclo se ejecuta n veces y dentro de su cuerpo se ejecuta una suma, que consideraremos como la operación elemental. De modo que nuestra recurrencia queda como:

$$T(n) = \begin{cases} 1 & n = 0 \\ T(n-1) + n & e.o.c. \end{cases} \quad (1.6)$$

2. Sustituir suficientes veces para encontrar una expresión en términos de un índice i .

Realizaremos el trabajo de sustituir para $n - 1$ y para $n - 2$ a la espera de deducir un patrón:

$$T(n) = T(n-1) + n$$

$$T(n-1) = T(n-2) + (n-1)$$

$$T(n) = T(n-2) + (n-1) + n \quad \text{sustituimos } T(n-1) \text{ por la expresión de arriba}$$

$$T(n-2) = T(n-3) + (n-2)$$

$$T(n) = T(n-3) + (n-2) + (n-1) + n \quad \text{sustituimos } T(n-2) \text{ por la expresión de arriba}$$

Observando las distintas expresiones para $T(n)$ encontramos que se va formando una suma de términos consecutivos, por ejemplo, en la última expresión para $T(n)$ nos encontramos con la suma $(n-2) + (n-1) + n$.

De modo que, en general, para un índice i podríamos escribir:

$$T(n) = T(n-i) + \sum_{k=0}^{i-1} (n-k) \quad (1.7)$$

3. *Escribir dicha expresión para el caso base:*

A partir de 1.7 tenemos que encontrar una expresión que incluya al caso base, es decir a $T(1)$, cosa que logramos si sustituimos i por $n-1$ para obtener:

$$T(n) = T(n-(n-1)) + \sum_{k=0}^{n-2} (n-k) \quad (1.8)$$

Prestando atención a la suma de la izquierda, podemos ver que al expandirla nos queda:

$$\begin{aligned} \sum_{k=0}^{n-2} (n-k) &= n + (n-1) + \dots + (n-(n-3)) + (n-(n-2)) \\ &= n + (n-1) + \dots + 3 + 2 \\ &= 2 + 3 + \dots + n \\ &= \sum_{k=2}^n k \end{aligned}$$

De modo que si reescribimos la expresión 1.7 tendremos un enunciado más amigable:

$$T(n) = T(1) + \sum_{k=2}^n k = 1 + \sum_{k=2}^n k = \sum_{k=1}^n k \quad (1.9)$$

Gracias a Gauss, sabemos que:

$$\sum_{k=1}^n k = \frac{n(n+1)}{2}$$

De modo que nuestra función de complejidad queda como sigue:

$$T(n) = \frac{n(n+1)}{2}$$

Algoritmo 5 Hoo (n)

Precondiciones: n es un entero, $n \geq 0$

```
1: if  $n \leq 1$  then  
2:   return 1  
3: else  
4:   return ( $Hoo(n-1) + Hoo(n-1)$ )  
5: end if
```

La intuición nos dice que esta vez $T(n) \in O(n^2)$. Queda como tarea moral encontrar las constantes $c \in \mathbb{R}^+$ y $n_0 \in \mathbb{N}$ que nos permitan probarlo.

En este tercer ejemplo se realizan dos llamadas recursivas cuyos resultados se suman. ¡Obtenemos su función de complejidad!

1. *Definir la recurrencia*

$$T(n) = \begin{cases} 1 & n = 1 \\ 2T(n-1) + 1 & n > 1 \end{cases} \quad (1.10)$$

2. *Sustituir suficientes veces para encontrar una expresión en términos de un índice i .*

$$\begin{aligned} T(n) &= 2T(n-1) + 1 \\ T(n-1) &= 2T(n-2) + 1 \\ T(n) &= 4T(n-2) + 3 && \text{sustituyendo } T(n-1) \text{ por la expresión de arriba} \\ T(n-2) &= 2T(n-3) + 1 \\ T(n) &= 8T(n-3) + 7 && \text{sustituyendo } T(n-2) \text{ por la expresión de arriba} \end{aligned}$$

Si reescribimos las expresiones anteriores para $T(n)$ como sigue:

$$\begin{aligned} T(n) &= 2T(n-1) + 1 = 2^1T(n-1) + 2^1 - 1 \\ T(n) &= 4T(n-2) + 3 = 2^2T(n-2) + 2^2 - 1 \\ T(n) &= 8T(n-3) + 7 = 2^3T(n-3) + 2^3 - 1 \end{aligned}$$

Se vuelve evidente que la expresión en general de $T(n)$ para un índice i es:

$$T(n) = 2^iT(n-i) + 2^i - 1 \quad (1.11)$$

3. *Escribir dicha expresión para el caso base*

Ahora bien, ya que queremos escribir la función para $T(n)$ en términos del caso base $T(1)$, necesitamos un valor para i que cumpla: $n - i = 1$.

Despejando, obtenemos: $i = n - 1$. Y al sustituir este valor en 1.11, obtenemos:

$$\begin{aligned}
T(n) &= 2^{n-1}T(1) + 2^{n-1} - 1 \\
&= 2^{n-1} + 2^{n-1} - 1 \\
&= 2(2^{n-1}) - 1 \\
&= 2^n - 1
\end{aligned}$$

Así, nuestra función de complejidad es $T(n) = 2^n - 1$.

Es claro que la función $T(n) = 2^n - 1$ es la función 2^n desplazada una unidad hacia abajo. Por tanto, para todo $n \in \mathbb{N}$ se cumple que $T(n) \leq 2^n$, esto quiere decir que $T(n) \in O(2^n)$.

MaxSum3

En la nota 3, introdujimos el algoritmo **MaxSum3** como una de las soluciones al problema de, dado un arreglo de números reales, encontrar el subarreglo de suma máxima. A continuación tenemos el pseudocódigo nuevamente:

Algoritmo 6 MAXSUM3(v, n)

1: **return** SUMPARTICION($v, 0, n - 1$)

Algoritmo 7 SUMPARTICION(v, inf, sup)

```

1: if  $inf > sup$  then
2:   return 0
3: end if
4: if  $inf = sup$  then
5:   return MAXIMO(0,  $v[inf]$ )
6: end if
7:  $m \leftarrow (inf + sup)/2$ 
8:  $sumactual \leftarrow 0$ 
9:  $izqmax \leftarrow 0$ 
10: for  $i \in \{m, \dots, inf\}$  do
11:    $sumactual \leftarrow sumactual + v[i]$ 
12:   if  $sumactual > izqmax$  then
13:      $izqmax \leftarrow sumactual$ 
14:   end if
15: end for
16:  $sumactual \leftarrow 0$ 
17:  $dermax \leftarrow 0$ 
18: for  $i \in \{m + 1, \dots, sup\}$  do
19:    $sumactual \leftarrow sumactual + v[i]$ 
20:   if  $sumactual > dermax$  then
21:      $dermax \leftarrow sumactual$ 
22:   end if
23: end for
24:  $sumainterseccion \leftarrow izqmax + dermax$ 
25:  $maxpartizq \leftarrow$  SUMPARTICION( $v, inf, m$ )
26:  $maxpartder \leftarrow$  SUMPARTICION( $v, m + 1, sup$ )
27: return Maximo( $sumainterseccion$ , Maximo( $maxpartizq, maxpartder$ ))

```

Partiendo de programar el algoritmo y realizar varias ejecuciones obtuvimos una gráfica que sugería una función de complejidad lineal. Sin embargo, gracias a las herramientas que hemos desarrollado hasta el momento, vamos a deducir su verdadera función de complejidad y concluir que su orden no es $O(n)$ sino $O(n \log n)$:

Siguiendo los pasos que habíamos introducido anteriormente, empecemos por definir una expresión de recurrencia.

En el algoritmo *SumParticion*, el caso recursivo presenta dos ciclos **for**. Dado que cada uno recorre la mitad de la arreglo de entrada, entonces, en conjunto, su ejecución toma un total de kn operaciones, donde k es una constante y considerando que n es el tamaño del arreglo de entrada. Posteriormente hay dos llamadas recursivas con la mitad del arreglo de entrada original, por lo que podemos deducir la siguiente expresión:

$$T(n) \begin{cases} c & n \leq 1 \\ 2T(n/2) + kn & n \geq 1 \end{cases} \quad (1.12)$$

Ahora busquemos resolver la recurrencia por sustitución:

$$\begin{aligned} T(n) &= 2T(n/2) + kn \\ T(n/2) &= 2T(n/2^2) + (n/2)k \\ T(n) &= 2^2T(n/2^2) + 2kn \\ T(n/2^2) &= 2^2T(n/2^3) + (n/2^2)k \\ T(n) &= 2^3T(n/2^3) + 3kn \end{aligned}$$

A partir de las expresiones que hemos obtenido para $T(n)$ podemos deducir la siguiente expresión general en términos de un índice i :

$$T(n) = 2^i T(n/2^i) + ikn \quad (1.13)$$

Y cuando queremos escribir $T(1)$ del lado izquierdo necesitamos que el índice i sea $\log_2 n$, pues $T(n/2^{\log_2 n}) = T(1)$. Así tenemos que:

$$\begin{aligned} T(n) &= 2^{\log_2 n} T(1) + kn \log_2 n \\ T(n) &= nT(1) + kn \log_2 n \\ T(n) &= cn + kn \log_2 n \end{aligned} \quad \text{ya que } T(1) = c$$

De modo que $T(n) = cn + kn \log_2 n$ es nuestra función de complejidad.

En la nota 4 vimos la propiedad de la suma que establecía el orden de la suma de dos funciones de complejidad como el máximo de sus órdenes. Es decir, cuando teníamos un par de funciones f_1 y f_2 cuyos ordenes de complejidad fuesen $O(g)$ y $O(h)$ respectivamente, se cumple que $f_1 + f_2 \in O(\max(g, h))$.

Ya que $cn \in O(n)$ y $kn \log_2 n \in O(n \log n)$, tenemos que:

$$T(n) = cn + kn \log_2 n \in O(\max(n, n \log n)) = O(n \log n)$$

En conclusión, el orden de $T(n)$ es $O(n \log n)$.



Figura 2: Suficiente trabajo por hoy.

Referencias

- [1] María del Carmen Gómez Fuentes and Jorge Cervantes Ojeda. *Introducción al Análisis y al Diseño de Algoritmos*. Universidad Autónoma Metropolitana, jun 2014.